

[AP 16] - EL CONTENEDOR DESCONOCIDO

Formas parte del equipo de ciberseguridad encargado de administrar un servidor Ubuntu Server que utiliza Docker.

El servidor aloja un servicio web corporativo mediante un contenedor.

El administrador responsable informa de que solamente debería existir el siguiente servicio:

Propiedad	Detalle
Nombre:	web-corporativa
Función:	Servidor web
Puerto externo:	8080

Sin embargo, durante una revisión rutinaria se ha detectado un aumento extraño en el consumo de recursos del servidor.

Tu objetivo será analizar el entorno Docker, identificar cualquier contenedor no autorizado y realizar una primera respuesta ante el incidente.

Objetivos

Durante la actividad deberás demostrar que sabes:

- Identificar imágenes y contenedores.
- Diferenciar contenedores activos y detenidos.
- Interpretar el estado de los contenedores.
- Identificar los puertos publicados.
- Consultar procesos.
- Consultar logs.
- Analizar el consumo de recursos.
- Inspeccionar un contenedor.
- Detener un contenedor sospechoso.
- Eliminarlo cuando la investigación haya terminado.

Preparación del laboratorio

Antes de que comience la actividad, debes crear dos contenedores:

```
docker run -d --name web-corporativa \ -p 8080:80 nginx:alpine
docker run -d --name worker-17 --cpus="0.25" alpine sh -c 'while true; do ;; done'
```

Genera algunas peticiones al primer servidor:

```
curl http://localhost:8080
curl http://localhost:8080
curl http://localhost:8080/no-existe
```

Comprobar:

```
docker ps
```

Deberían existir:

- web-corporativa
- worker-17

Nota: No explicar al alumnado cuál es el contenedor sospechoso.
La única información que deben recibir es: **El único contenedor autorizado es web-corporativa.**

Escenario

Son las 10 horas.

El sistema de monitorización ha informado de un consumo de CPU superior al habitual.

El responsable del servidor confirma que únicamente debería existir un contenedor denominado: web-corporativa

El equipo de ciberseguridad solicita que realices una primera investigación.

No debes eliminar ningún elemento hasta haber recopilado evidencias suficientes.

Fase 1 — Inventario del sistema

- Consulta la versión de Docker: [Evidencia visual adjunta en imágenes]
- Comprueba que Docker está funcionando: [Evidencia visual adjunta en imágenes]
- Consulta las imágenes disponibles: [Evidencia visual adjunta en imágenes]
- Consulta los contenedores activos: [Evidencia visual adjunta en imágenes]
- Consulta todos los contenedores: [Evidencia visual adjunta en imágenes]

Salida del comando docker ps -a ejecutado en el sistema:

```
server@ubuntuserver:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        NAMES
STATUS        PORTS          NAMES
a4974445b67e   alpine        "sh -c 'while true; ..." 7 minutes ago  Up 7 minutes
worker-17
c378032df66a   nginx:alpine  "/docker-entrypoint. ..." 7 minutes ago  Up 7 minutes
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp web-corporativa
caf6cefe40c6   nginx:alpine  "/docker-entrypoint. ..." 3 hours ago    Up 2 hours
0.0.0.0:80->80/tcp, [::]:80->80/tcp web-inicial
af7cb001dd39   ubuntu:24.04  "bash"                   3 days ago     Exited (0) 3 days ago
labubuntu
0ad3a66e6a85   hello-world   "/hello"                 3 days ago     Exited (0) 3 days ago
flamboyant_cerf
```

Salida del comando docker ps ejecutado en el sistema:

```
server@ubuntuserver:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  NAMES        CREATED        STATUS
PORTS          NAMES
```

```

a4974445b67e  alpine          "sh -c 'while true; ..."  5 minutes ago  Up 5
minutes                                             worker-17
c378032df66a  nginx:alpine    "/docker-entrypoint...." 6 minutes ago  Up 6 minutes
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  web-corporativa
caf6cfe40c6   nginx:alpine    "/docker-entrypoint...." 3 hours ago    Up 2 hours
0.0.0.0:80->80/tcp, [::]:80->80/tcp  web-inicial

```

Salida del comando `docker images` ejecutado en el sistema:

```

server@ubuntuserver:~$ docker images

IMAGE                ID                DISK USAGE    CONTENT SIZE    EXTRA    In Use
alpine:latest        28bd5fe8b56d     13.6MB        4.27MB          U
hello-world:latest   c3cbe1cc1aa5     22.6kB        10.3kB          U
nginx:alpine         4a73073bd557     92.8MB        26.9MB          U
ubuntu:24.04         4fbb8e6a8395     141MB         30.8MB          U

```

Tabla de Inventario Completa

Contenedor	Imagen	Estado	Puerto
worker-17	alpine	Up 7 minutes	-
web-corporativa	nginx:alpine	Up 7 minutes	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
web-inicial	nginx:alpine	Up 2 hours	0.0.0.0:80->80/tcp, [::]:80->80/tcp
labubuntu	ubuntu:24.04	Exited (0) 3 days ago	-
flamboyant_cerf	hello-world	Exited (0) 3 days ago	-

Fase 2 — Identificación de activos autorizados

¿Responde el servidor web?

Sí, responde correctamente al comando curl realizado sobre el puerto configurado.

¿Qué código HTTP devuelve?

Devuelve `HTTP/1.1 200 OK` tal y como se observa en la traza obtenida.

Fase 3 — Detección y análisis de anomalías

Ejecuta:

```
docker stats --no-stream
```

Registro de métricas obtenido:

```

a4974445b67e  worker-17          25.25%    352KiB / 4.036GiB    0.01%    1.12kB /
126B         0B / 0B           1

```

¿Qué contenedor utiliza más CPU?

worker-17, lo cual implica que algo pasa ahí porque es demasiado alta y no es la esperada por la empresa.

¿Coincide con el contenedor no autorizado?

Sí, porque la empresa dice que solo debería existir `web-corporativa`.

¿Podemos afirmar únicamente por un consumo elevado de CPU que un contenedor es malicioso? Justifica brevemente la respuesta.

En este caso, `worker-17` es sospechoso porque combina dos indicadores: un consumo de CPU anómalo (25,25%) y la existencia de un contenedor no autorizado respecto a la política de la empresa (solo debería existir `web-corporativa`). Sin embargo, harían falta evidencias adicionales antes de clasificarlo definitivamente como malicioso.

Fase 4 — Análisis de procesos

¿Qué diferencias observas entre los procesos de ambos contenedores?

Mucha más CPU en `worker-17`, pero menos memoria y net, solo 1 PIDS y en block i/o también menos. Se observa un consumo elevado de CPU con un único proceso activo, mientras que los contenedores `web` no muestran actividad relevante.

¿Qué comando está ejecutando el contenedor sospechoso?

```
server@ubuntuserver:~$ docker top worker-17
UID          PID          PPID         C
STIME        TTY          TIME         CMD
root         3053         3030         25
16:30        ?           00:04:51     sh -c while true; do ;;
done
```

El comando raro es: `sh -c while true; do ;; done`

Fase 5 — Inspección del contenedor

Docker proporciona mediante `inspect` información detallada de la configuración y el estado de los contenedores.

Obtén información detallada de `worker-17` ejecutando `docker inspect worker-17`.

¿Qué imagen utiliza?

```
"Image": "sha256:28bd5fe8b56d1bd048e5babf5b10710ebe0bae67db86916198a6eec434943f8b"
(Imagen base Alpine).
```

¿Qué comando ejecuta?

```
"Cmd": [
  "sh",
  "-c",
  "while true; do ;; done"
]
```

¿El contenedor está actualmente activo?

```
"State": {
  "Status": "running",
  "Running": true,
  "Paused": false,
  "Restarting": false,
  "Dead": false
}
```

Sí, está activo (running).

Fase 6 — Análisis del contenedor legítimo

Ahora analizaremos el servidor web. Consulta sus logs:

```
docker logs web-corporativa
```

Traza de logs obtenida:

```
172.17.0.1 - - [20/Jul/2026:16:30:56 +0000] "GET / HTTP/1.1" 200 896 "-" "curl/8.18.0"
172.17.0.1 - - [20/Jul/2026:16:30:56 +0000] "GET / HTTP/1.1" 200 896 "-" "curl/8.18.0"
172.17.0.1 - - [20/Jul/2026:16:30:56 +0000] "GET /no-existe HTTP/1.1" 404 153 "-"
"curl/8.18.0" "-"
2026/07/20 16:30:56 [error] 30#30: *3 open() "/usr/share/nginx/html/no-existe" failed
(2: No such file or directory), client: 172.17.0.1, server: localhost, request: "GET /
no-existe HTTP/1.1", host: "localhost:8080"
172.17.0.1 - - [20/Jul/2026:16:42:10 +0000] "HEAD / HTTP/1.1" 200 0 "-" "curl/8.18.0"
172.17.0.1 - - [20/Jul/2026:16:42:10 +0000] "HEAD / HTTP/1.1" 200 0 "-" "curl/8.18.0"
```

¿Aparecen peticiones HTTP?

Sí, aparecen reflejadas en los registros internos tras usar el comando.

¿Existe alguna petición que haya producido un error?

```
"GET /no-existe HTTP/1.1" 404 153 "-" "curl/8.18.0"
```

El servidor intenta acceder al recurso: `/usr/share/nginx/html/no-existe` pero el archivo no existe: `open() "/usr/share/nginx/html/no-existe" failed (2: No such file or directory)`. Esto corresponde a un error de recurso no encontrado, no a un fallo del servidor.

¿Por qué pueden ser importantes los logs durante un incidente de seguridad?

Principalmente por la trazabilidad que nos puede ayudar a monitorizar y a dar respuesta a incidentes y a hacer análisis forense para establecer mejora continua mejorar el DCP y el BCP dentro del PDCA.

Fase 7 — Acceso al contenedor legítimo

Ejecuta:

```
docker exec web-corporativa hostname
docker exec web-corporativa whoami

docker exec -it web-corporativa sh
hostname
whoami
ps
```

Al ejecutar `docker exec web-corporativa hostname` se obtiene el nombre del host interno del contenedor. Con `docker exec web-corporativa whoami` se comprueba el usuario con el que se ejecutan los procesos dentro del contenedor. Después, al entrar de forma interactiva con `docker exec -it web-corporativa sh` y ejecutar los comandos locales, se observa información del entorno interno del contenedor y los procesos activos.

¿Se ha detenido web-corporativa al ejecutar exit?

No.

Explica brevemente por qué:

Al ejecutar `exit` únicamente se cierra la sesión interactiva de la shell abierta mediante `docker exec`. El contenedor `web-corporativa` continúa funcionando. Para detener realmente el contenedor habría que usar `docker stop web-corporativa`.

| Fase 8 — Contención del incidente

La investigación ha determinado el servidor web es: **AUTORIZADO**

Estructura de Incidentes:

- **web-corporativa** → AUTORIZADO
- **worker-17** → NO AUTORIZADO

El primer objetivo de la respuesta ante el incidente será detener la actividad sospechosa. Y luego comprobar que todo está correcto. Primero hay que salir del contenedor `web-corporativa` de su shell haciendo `exit` e ir al host `docker`.

¿Continúa ejecutándose worker-17?

Tiramos `docker ps` y vemos que aún sigue activo:

```
a4974445b67e    alpine          "sh -c 'while true; ..."    39 minutes ago    Up 39
minutes                                                worker-17
```

Para pararlo tiramos: `docker stop worker-17`

¿Continúa existiendo el contenedor?

Hacemos `docker ps` y ya no está activo. Para verificar mejor hacemos `docker ps -a` que sale como **exited** en status.

Sí, sigue existiendo pero está parado.

¿Por qué puede ser interesante detener un contenedor antes de eliminarlo?

Detener un contenedor antes de eliminarlo es importante porque permite:

- Preservar evidencias para una investigación posterior.
- Revisar información del contenedor (`docker inspect`).
- Analizar logs (`docker logs worker-17`).
- Examinar la configuración, imagen y procesos antes de perderlos.
- Evitar destruir información útil para determinar el origen del incidente.

| Fase 9 — Comprobación del servicio legítimo

Después de contener el incidente debemos comprobar que el servicio autorizado continúa funcionando.

¿Continúa funcionando la web corporativa?

Con el `docker ps -a` hemos visto que sigue activo (Up) la `web-corporativa`:

```
c378032df66a    nginx:alpine    ...    Up 41 minutes    0.0.0.0:8080->80/tcp    web-
corporativa
```

Esto indica que el contenedor sigue en estado Up, el servicio nginx sigue ejecutándose, y el puerto 8080 continúa publicado y accesible desde el host.

¿Qué contenedores aparecen ahora en ejecución?

- c378032df66a web-corporativa
- caf6cefe40c6 web-inicial

El contenedor sospechoso a4974445b67e worker-17 se encuentra en estado Exited (137), por lo que ya no está en ejecución.

| Fase 10 — Erradicación

Una vez recopiladas las evidencias y contenido el incidente, eliminamos el contenedor no autorizado y comprobamos.

¿Continúa existiendo worker-17?

Sí, en este momento todavía existe. La evidencia es `docker ps -a`, donde aparece:

```
a4974445b67e    alpine    "sh -c 'while true; ...'"    Exited (137)    worker-17
```